Phase 4: PySpark Processing Layer Completion Report

1. Phase Overview

Phase 4 of the Banking Data Governance & Data Quality Pipeline focused on building the PySpark processing layer.

The goal of this phase was to read raw Parquet files generated by the Phase 3 Python ingestion layer, apply standardization and schema handling using PySpark, and write clean standardized Parquet outputs to a new standardized data zone.

This phase represents the first Spark-based transformation step in the project. It moves the pipeline from simple raw data extraction into structured, governed, and schema-aware data processing.

The Phase 4 processing flow is:

PostgreSQL banking source system
→ Python ingestion layer
→ data/raw/
→ PySpark processing layer
→ data/standardized/

Phase 4 did not include curated fact and dimension modeling, SQL validation scripts, final data quality rule execution, OpenMetadata setup, governance YAML artifacts, lineage documentation, or Agile/JIRA simulation artifacts. Those items remain planned for later phases.

---

2. Phase 4 Objective

The objective of Phase 4 was:

To use PySpark to read raw Parquet files from the raw zone, apply basic cleaning, standardization, schema handling, and type normalization, and write standardized Parquet outputs partitioned by load date.

The expected input pattern from Phase 3 was:

data/raw/<table_name>/load_date=YYYY-MM-DD/<table_name>.parquet

The expected output pattern for Phase 4 was:

data/standardized/<table_name>/load_date=YYYY-MM-DD/<table_name>.parquet

Example:

data/standardized/customers/load_date=2026-06-16/customers.parquet
data/standardized/accounts/load_date=2026-06-16/accounts.parquet
data/standardized/transactions/load_date=2026-06-16/transactions.parquet

In local PySpark execution, the .parquet output path is treated as a Spark output directory containing Parquet part files.

---

3. Phase 4 Scope

Phase 4 included only the PySpark raw-to-standardized processing layer.

In Scope

Phase 4 included:

- PySpark project structure
- Reusable Spark session utility

- Raw Parquet reading by load date
- Table whitelist validation
- Load-date validation
- Basic column standardization
- String trimming
- Blank-string-to-null handling
- Email lowercasing
- Code/status/type/category/source-system uppercasing
- Metadata column creation
- Table-specific schema configuration
- Required-column validation
- Table-specific type casting
- Preferred output column ordering
- Standardized Parquet writing
- Row-count verification
- Summary CSV reporting
- README command updates
- Git commit and phase tag

Out of Scope

The following items were intentionally not included in Phase 4:

- Curated fact and dimension models
- Final warehouse modeling
- SQL data quality validation scripts
- PySpark data quality rule execution
- OpenMetadata setup
- Governance YAML artifacts
- Business glossary
- Lineage mapping
- Agile/JIRA simulation files

This scope control kept Phase 4 focused on PySpark processing only.

---

4. Tables Processed

Phase 4 processed the same 10 approved source/control tables created in Phase 2 and extracted in Phase 3:

1. branches
2. products
3. reference_codes
4. customers
5. accounts
6. transactions
7. customer_risk_scores
8. data_quality_rules

9. stewardship_issues
10. source_to_target_mappings

The job validates requested tables against this approved list before processing.

---

5. Phase 4 Repository Structure

Phase 4 added the following Spark structure:

```
spark/
 jobs/
  process_raw_to_standardized.py
 utils/
  spark_session.py
  schema_config.py
```

Runtime-generated folders:

```
data/
 standardized/

logs/
 spark/
```

The runtime folders are generated locally and are intentionally ignored by Git.

---

6. Files Created or Updated

Files Created

spark/utils/spark_session.py
spark/utils/schema_config.py
spark/jobs/process_raw_to_standardized.py

Files Updated

.gitignore
README.md
requirements.txt

Runtime Folders Generated Locally

data/standardized/
logs/spark/

These runtime folders were not committed to Git.

---

7. Phase 4A: Baseline PySpark Raw-to-Standardized Processing

Phase 4A Goal

The goal of Phase 4A was to create the first working PySpark job that reads raw Parquet files, applies generic standardization, writes standardized Parquet outputs, and produces row-count summary evidence.

Phase 4A Work Completed

Phase 4A added:

spark/utils/spark_session.py

spark/jobs/process_raw_to_standardized.py
It also updated:
.gitignore
README.md
requirements.txt

Phase 4A Capabilities

Phase 4A implemented:

- Local Spark session utility
- Command-line PySpark job
- --load-date argument
- --input-dir argument
- --output-dir argument
- --log-dir argument
- --tables argument
- Load-date validation using YYYY-MM-DD
- Approved-table validation
- Raw Parquet reading
- Standardized Parquet writing
- Lowercase snake_case column standardization
- String trimming
- Blank string conversion to null
- Email field lowercasing
- Code/status/type/category/source-system uppercasing
- Metadata column creation
- Row-count comparison
- Timestamped summary CSV generation
- Non-zero exit behavior when any table fails

Metadata Columns Added

Phase 4A added the following metadata columns to standardized outputs:
source_table
source_load_date
standardized_load_date
standardized_at_utc
These columns help preserve basic processing traceability from raw to standardized data.

Phase 4A Validation

The following validations passed:
java -version
pip install -r requirements.txt
python -m py_compile spark\utils\spark_session.py spark\jobs\process_raw_to_standardized.py
python spark\jobs\process_raw_to_standardized.py --tables customers --load-date 2026-06-16
python spark\jobs\process_raw_to_standardized.py --load-date 2026-06-16
Single-table validation succeeded for customers:

customers: raw 500, standardized 500, SUCCESS
The full run succeeded for all 10 tables.

Phase 4A Commit

9dde0d1 Phase 4A - Add PySpark raw to standardized processing

---

8. Phase 4B: Schema Handling and Type Normalization

Phase 4B Goal

The goal of Phase 4B was to harden the standardized layer by adding table-specific schema configuration, required-column checks, type casting, and preferred output column ordering.
This made the standardized layer more reliable and closer to a production-style data processing contract.

Phase 4B Work Completed

Phase 4B added:
spark/utils/schema_config.py
It also updated:
spark/jobs/process_raw_to_standardized.py
README.md

Phase 4B Capabilities

Phase 4B implemented:

- Table-specific schema configuration for all 10 approved tables
- Required-column checks after column standardization
- Type casting for IDs, amounts, balances, scores, dates, timestamps, booleans, and strings
- Preferred output column ordering
- Better failure handling for missing required columns
- Enhanced summary CSV fields
- README validation notes

Business Source Table Handling

Some source data already contained a business column named source_table.
To avoid conflict with the processing metadata column source_table, Phase 4B preserved business source_table values as:
source_table_name
The metadata column remained:
source_table
This avoided ambiguity and preserved both business context and pipeline metadata.

---

9. Standardized Output Design

The standardized zone uses local Parquet files partitioned by load date.
Output pattern:
data/standardized/<table_name>/load_date=YYYY-MM-DD/<table_name>.parquet
Example output paths:
data/standardized/branches/load_date=2026-06-16/branches.parquet
data/standardized/products/load_date=2026-06-16/products.parquet

data/standardized/reference_codes/load_date=2026-06-16/reference_codes.parquet
data/standardized/customers/load_date=2026-06-16/customers.parquet
data/standardized/accounts/load_date=2026-06-16/accounts.parquet
data/standardized/transactions/load_date=2026-06-16/transactions.parquet
data/standardized/customer_risk_scores/load_date=2026-06-16/customer_risk_scores.parquet
data/standardized/data_quality_rules/load_date=2026-06-16/data_quality_rules.parquet
data/standardized/stewardship_issues/load_date=2026-06-16/stewardship_issues.parquet
data/standardized/source_to_target_mappings/load_date=2026-06-16/source_to_target_mappings.parquet
This design creates a clean separation between the raw landing zone and the standardized processing zone.

---

10. Summary CSV Reporting

Phase 4 writes a timestamped summary CSV under:
logs/spark/
Example file pattern:
logs/spark/raw_to_standardized_summary_YYYYMMDD_HHMMSS.csv
The final summary CSV includes:
table_name
status
raw_row_count
standardized_row_count
raw_column_count
standardized_column_count
missing_required_columns
input_path
output_path
error_message
This provides lightweight operational evidence for each Spark processing run.

---

11. Phase 4B Validation

The following validation commands were run successfully:
python -m py_compile spark\utils\spark_session.py spark\utils\schema_config.py spark\jobs\process_raw_to_standardized.py
python spark\jobs\process_raw_to_standardized.py --load-date 2026-06-16
All 10 tables succeeded.

Final Row Count Validation

branches SUCCESS 10 10
products SUCCESS 6 6
reference_codes SUCCESS 72 72
customers SUCCESS 500 500
accounts SUCCESS 990 990

transactions SUCCESS 5000 5000
customer_risk_scores SUCCESS 742 742
data_quality_rules SUCCESS 12 12
stewardship_issues SUCCESS 8 8
source_to_target_mappings SUCCESS 25 25
Validation confirmed:

- Python files compiled successfully
- Spark job ran successfully
- All 10 approved tables were processed
- Raw row counts matched standardized row counts
- Required columns were present
- Standardized Parquet outputs were written
- Summary CSV was generated
- Runtime folders were ignored by Git
- Final Git status was clean

A local Windows winutils.exe warning appeared during Spark execution, but the job completed successfully. This warning did not affect the pipeline output.

---

12. Git History After Phase 4

Final Git history after completing and tagging Phase 4:
7cdf802 (HEAD -> main, tag: phase-4-complete) Phase 4B - Add schema handling for standardized Spark layer
9dde0d1 Phase 4A - Add PySpark raw to standardized processing
8554b4b (tag: phase-3-complete) Phase 3B - Harden raw ingestion validation
5a8e5d4 Phase 3A - Add PostgreSQL raw Parquet ingestion
ef5944f (tag: phase-2-complete) Phase 2D - Add source data load instructions and compose cleanup
Phase 4 tag:
phase-4-complete
Current tags:
phase-1-complete
phase-2-complete
phase-3-complete
phase-4-complete

---

13. Final Phase 4 Status

Phase 4 is complete.

Final Status

Item	Status
Phase 4A baseline Spark processing	Complete
Phase 4B schema handling	Complete
Spark session utility	Complete

Item	Status
Raw Parquet reading	Complete
Standardized Parquet writing	Complete
Load-date partitioning	Complete
Table whitelist validation	Complete
Load-date validation	Complete
Column standardization	Complete
String cleanup	Complete
Metadata columns	Complete
Required-column checks	Complete
Type casting	Complete
Preferred column ordering	Complete
Summary CSV reporting	Complete
Runtime folders ignored by Git	Complete
Git commit completed	Complete
Phase tag created	Complete

---

14. Skills Practiced in Phase 4

Phase 4 helped practice the following skills:

- PySpark development
- SparkSession creation
- Parquet file processing
- Data lake layer design
- Raw-to-standardized processing
- Schema handling
- Type casting
- Column standardization
- String cleanup
- Metadata column creation
- Load-date partitioning
- Row-count reconciliation
- Required-column validation
- Runtime logging and summary reporting
- Command-line argument handling
- Git commit and tagging discipline
- SDLC-style phased delivery

---

15. Interview Explanation

A clear interview explanation for Phase 4 would be:

“In Phase 4, I built the PySpark processing layer for the banking governance pipeline. The previous phase landed PostgreSQL source data into a raw Parquet zone partitioned by load date. In Phase 4, I created a Spark job that reads those raw Parquet files, validates the load date and approved table list, standardizes column names, cleans string fields, normalizes email and code/status/type columns, and writes the results into a standardized Parquet zone. I also added schema configuration for all 10 banking source and control tables, including required-column checks, table-specific type casting, preferred output ordering, and metadata columns for traceability. The job generates a summary CSV with row counts, column counts, missing required columns, status, and error messages. All 10 tables processed successfully, and raw row counts matched standardized row counts.”

---

16. Phase 4 Conclusion

Phase 4 successfully created the PySpark processing foundation for the Banking Data Governance & Data Quality Pipeline.

The project now has a working flow from:

PostgreSQL banking source system
→ Python ingestion layer
→ raw Parquet zone
→ PySpark standardized processing layer
→ standardized Parquet zone

This prepares the project for the next phase:

Phase 5: Curated Warehouse Modeling

In Phase 5, the standardized data can be used to build curated fact and dimension outputs, including customer, account, branch, product, transaction, and risk-related warehouse models.